

Assignment Report on

RELIABLE UDP COMMUNICATION IN MININET

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to implement reliable communication over UDP in Mininet using the Stop-and-Wait ARQ protocol. UDP does not guarantee delivery, ordering, or retransmission. Therefore, reliability is added at the application layer by using sequence numbers, acknowledgements (ACKs), timeout, and retransmissions.

In this assignment, a UDP client and a UDP server are created in Mininet using the topology **h1 ---- s1 ---- h2**, where h1 acts as the server and h2 acts as the client. The client sends 10 messages to the server, one at a time. Each message contains a sequence number, and the server replies with an ACK. If the client does not receive the ACK within the timeout period, it retransmits the packet. In this way, reliable delivery is achieved over UDP.

2. Mininet Topology and Setup

The Mininet topology used for a simple two-host network connected by one switch:

h1 ---- s1 ---- h2

Role of each node:

- **h1** → UDP Server
- **h2** → UDP Client
- **s1** → Switch connecting both hosts

Before running the client-server program, the topology was verified using the Mininet commands: nodes, net, and pingall. These commands confirm that the hosts and switch are present, the topology is connected correctly, and both hosts can communicate successfully.

3. Timeout Rule Used

According to the assignment instructions, the timeout value must be selected according to the last digit of the roll number.

0-1 → 0.5 s
2-3 → 0.7 s
4-5 → 1.0 s
6-7 → 1.2 s
8-9 → 1.5 s

My timeout value:

Roll Number: 22BCS104

Last Digit: 4

Timeout Used: 1.0 seconds

This timeout is used by the client while waiting for an ACK from the server.

4. Packet Format and ACK Format

The client sends each message in the following format:

SEQ|MESSAGE

Example:

3|Message 3 from h2

The server replies with an acknowledgement in the following format:

ACK|SEQ

Example:

ACK|3

This ACK confirms that the server has received the packet with sequence number 3.

5. Working of the Program

Client side working:

1. A UDP socket is created.
2. The timeout value is selected automatically from the roll number.
3. The client sends 10 messages to the server, one packet at a time.
4. Each packet is sent in the format SEQ|MESSAGE.
5. After sending a packet, the client waits for an ACK from the server.
6. If the ACK is received within the timeout period and it matches the expected sequence number, the client sends the next packet.
7. If the ACK is not received before timeout, the client retransmits the same packet.
8. The client keeps track of total messages, total packets sent, total retransmissions, and transfer time.
9. After all 10 messages are delivered successfully, the client sends an END message to the server and writes the experiment results into result_table.csv.

Server side working:

1. A UDP socket is created and bound to port 5000.
2. The server waits continuously for packets from the client.
3. When a packet is received, the server extracts the sequence number and message text.
4. The server checks whether the sequence number has already been received.
5. If it is new, it is counted as a unique message. If it has already been received, it is counted as a duplicate.
6. The server sends an ACK for every received packet, including duplicates.
7. After receiving the END message from the client, the server prints the final statistics and stops.

6. Experimental Conditions

The reliable UDP program was tested under the following three Mininet conditions:

Case 1: 0% packet loss

```
sudo mn --link tc,bw=5,delay=30ms,loss=0
```

Case 2: 5% packet loss

```
sudo mn --link tc,bw=5,delay=30ms,loss=5
```

Case 3: 10% packet loss

```
sudo mn --link tc,bw=5,delay=30ms,loss=10
```

For each experiment, the server was run on h1, the client was run on h2, outputs were observed and screenshots were captured, and the results were recorded in result_table.csv.

7. Result Table

roll_no	name	Loss percent	timeout	Total messages	Total Packets sent	Total retransmissions	Transfer Time seconds	status
22BCS104	Ali	0	1.0	10	10	0	1.2094	SUCCESS
22BDD110	shahan	0	0.5	10	11	1	1.7131	SUCCESS
67BCs001	rashmi	0	0.5	10	16	6	4.2379	SUCCESS

8. Observation and Analysis

When packet loss is 0%, almost all packets and ACKs are delivered successfully on the first attempt. Therefore, retransmissions are very low or zero, total packets sent remains close to 10, and transfer time is minimum.

When packet loss increases to 5%, some packets or ACKs may be dropped by the network. In that case, the client does not receive ACK for some packets, retransmissions increase, total packets sent becomes greater than 10, and transfer time also increases.

When packet loss becomes 10%, packet drops occur more frequently. As a result, the number of retransmissions increases further, total packets sent becomes much larger than 10, and transfer time becomes highest among the three cases.

This behavior is expected because Stop-and-Wait depends on ACK reception for every packet. If the ACK or packet is lost, the sender must retransmit the packet after timeout. Therefore, as network loss increases, communication becomes slower and less efficient due to repeated retransmissions.

9. Answers to the Required Questions

1. Why is ACK needed?

ACK is needed to confirm that the server has successfully received a packet.

Since UDP itself does not guarantee delivery, the sender must know whether the packet reached the receiver.

2. Why is timeout needed?

Timeout is needed so that the client does not wait forever for an ACK. If the ACK is not received within the timeout period, the client assumes that either the packet or the ACK was lost and retransmits the same packet.

3. Why can duplicate messages occur?

Duplicate messages occur when the server receives a packet and sends an ACK, but the ACK is lost before reaching the client. Since the client does not receive the ACK, it retransmits the same packet. The server then receives that packet again and treats it as a duplicate.

4. What happens when packet loss increases?

When packet loss increases, more packets or ACKs are lost, retransmissions increase, total packets sent increases, and transfer time becomes larger.

5. How is this method similar to TCP?

This method is similar to TCP because it uses sequence numbers, acknowledgements, timeout, and retransmissions to provide reliable communication.

6. How is it different from TCP?

This method is different from TCP because it is a manually implemented reliability mechanism over UDP. It only provides basic Stop-and-Wait reliability, while TCP provides connection establishment, flow control, congestion control, sliding window transmission, and more advanced error recovery.

10. Screenshots Included

The following screenshots should be included in the submission folder:

1. **nodes.png** – output of nodes
2. **net.png** – output of net
3. **pingall.png** – output of pingall
4. **server_output.png** – server execution output
5. **client_output.png** – client execution output

These screenshots prove that the Mininet topology was created correctly, connectivity between hosts was verified, and the reliable UDP mechanism worked as expected.

11. Conclusion

This assignment demonstrates how reliability can be added to UDP communication using Stop-and-Wait ARQ. Since UDP does not provide guaranteed delivery, the client and server were designed to use sequence numbers, acknowledgements, timeout, and retransmissions to ensure successful delivery of all messages.

The results show that the client successfully delivers all 10 messages to the server, the server correctly detects duplicate packets, retransmissions increase when packet loss increases, and transfer time also increases as the loss percentage becomes higher.

Hence, this assignment successfully illustrates the basic concept of reliable data transfer over an unreliable transport protocol and provides a practical understanding of how reliability mechanisms similar to TCP can be built on top of UDP.